

Project Two: Summary and Reflections Report

During Project One, I developed and tested three back-end services for a mobile application: the contact service, task service, and appointment service. My overall testing approach was to use JUnit 5-unit tests to make sure each class, and service followed the software requirements. I tested both valid and invalid inputs instead of only checking whether the program worked under normal conditions. This helped me verify that the application could correctly handle expected inputs as well as incorrect or unexpected data. For contact service, I focused my tests on the requirements for the contact ID, first name, last name, phone number, and address. For example, the contact ID could not be null or longer than 10 characters, and the phone number had to contain exactly 10 digits. I tested valid contact creation as well as invalid conditions such as null values, values that exceeded the maximum length, and incorrectly formatted phone numbers. I also tested the “ContactService” methods for adding, updating, and deleting contacts and made sure duplicate contact IDs were rejected.

I followed a similar approach to task service. The task ID was required to be unique, not null, no longer than 10 characters, and unable to be updated. The task name could not be null or longer than 20 characters, while the description could not be null or longer than 50 characters. My JUnit tests checked valid task creation along with invalid IDs, names, and descriptions. I also tested the service methods responsible for adding, updating, and deleting tasks. By testing both the Task class and “TaskService” class, I was able to check the validation requirements as well as the operations performed on stored tasks. Last, For the appointment service, I tested the requirements for the appointment ID, appointment date, and description. The appointment ID could not be null or longer than 10 characters; the date could not be null or occur in the past, and the description could not be null or longer than 50 characters. One important part of my testing

was creating a future date in my setUp() method using “calendar.add(Calendar.DAY_OF_YEAR, 1);” This allowed the test to always create a valid future appointment instead of using a hard-coded date that could eventually become outdated. I also tested adding appointments, rejecting duplicate appointment IDs, handling null appointments, retrieving appointments, and deleting appointments. My testing approach was closely aligned with the software requirements because I used the requirements to determine which test cases needed to be created. Instead of testing only the parts of the program that I expected to work, I created tests specifically for the restrictions listed in the requirements. This gave me a way to verify that my implementation matched what the customer requested.

The overall quality of my JUnit tests was also demonstrated through code coverage. Across the contact, task, and appointment services, my tests achieved approximately 90% of code coverage. This showed that the majority of the classes, methods, validation statements, and service operations were executed during testing. I reached this level of coverage by testing both valid and invalid conditions, including null values, values exceeding maximum lengths, duplicate IDs, incorrectly formatted phone numbers, and invalid appointment dates. Although high code coverage does not automatically mean that every possible defect has been found, reaching approximately 90% gave me confidence that the most important requirements and code paths were being tested.

Writing the JUnit tests helped me better understand how closely software development and testing are connected. I ensured that my code was technically sound by testing both successful and unsuccessful operations. JUnit provides assertions that allow developers to compare expected results with actual program behavior and verify that expected exceptions occur (JUnit Team, n.d.). For example, one of my appointment service tests used:

`“assertEquals(appointment, service.getAppointment("1001"));`” This verified that after an appointment was added to the service, the correct appointment could be retrieved. I also used `“assertThrows()”` to verify that the program responded correctly when an operation violated a requirement. I also tried to make my testing code more efficient. One way I did this was through the JUnit `“@BeforeEach”` annotation. JUnit uses lifecycle methods such as `“@BeforeEach”` to execute setup code before individual test methods (JUnit Team, n.d.). Instead of creating a new service, appointment, and future date inside every individual test, I created these common objects in the `“setUp()”` method so they could be reused by each test. This reduced unnecessary duplicate code and made the tests easier to read and maintain. I also kept individual tests focused on specific behavior. When a test failed, this made it easier to identify which requirement or method was causing the problem.

The main software testing technique I used throughout Project One was unit testing. Unit testing focuses on testing individual components of software so that their behavior can be evaluated separately from the complete system. JUnit 5 provided the framework I used to create and automatically execute these tests. JUnit supports assertions and other testing features that allow developers to verify expected program behavior (JUnit Team, n.d.). This technique worked well for my project because the contact, task, and appointment classes each contained specific validation requirements that could be tested individually.

I also used positive, negative, and boundary testing. Positive testing involved entering valid information and confirming that the program accepted it and performed the expected operation. Negative testing involved intentionally providing invalid information, such as null values, duplicate IDs, strings that were too long, or appointment dates in the past. Boundary testing was especially useful because many of the requirements had exact character limits.

Boundary value analysis focuses on testing around the boundaries of an allowed range, where defects can often occur (International Software Testing Qualifications Board [ISTQB], n.d.). For example, an ID in my project could contain up to 10 characters. Testing values at and beyond that limit helped verify that acceptable values were allowed while invalid values were rejected.

There were other testing techniques that I did not use during this project. Integration testing focuses on interactions between components or systems, while system testing evaluates the behavior and capabilities of the complete system (ISTQB, n.d.). I also did not perform performance, security, or acceptance testing. Performance testing can evaluate characteristics such as response time and behavior under different workloads. Security testing focuses on determining whether a system adequately protects information and maintains expected security characteristics. Acceptance testing is used to determine whether a system meets acceptance criteria and is suitable for the needs of users or other stakeholders (ISTQB, n.d.).

Each of these techniques has different practical uses and implications. Unit testing is useful during development because defects can be identified within individual components before those components are combined with the rest of the application. Integration testing becomes especially important when an application uses databases, APIs, or multiple services that depend on one another. System and acceptance testing become more important when a complete application needs to be evaluated from the perspective of the overall system or customer. Performance testing would be especially useful for applications with large numbers of users or transactions. Security testing would be important for applications that handle private, financial, or otherwise sensitive information. Using the correct testing technique depends on the type of software being developed, its requirements, and the risks associated with failure.

One of the most important parts of my mindset while working on this project was caution. As a tester, I could not assume that my code was correct just because it was compiled or passed a basic test. I needed to think about how the code could fail and create tests for those situations. For example, an appointment could be successfully added to the service while there could still be a defect that allowed duplicate IDs or dates in the past. Testing these different situations helped me identify problems that might not appear during normal use. It was also important to appreciate the complexity and relationships between different parts of the code. A validation problem in one class could eventually cause problems somewhere else in the application. For example, if the Contact class allowed an incorrectly formatted phone number to be stored, another component could later attempt to use that incorrect information. Similarly, allowing a duplicate ID could make it difficult for another service to determine which record should be updated or deleted. Even though the services in this project were relatively small, the same type of problem could have a much larger effect on a real application with many connected components.

I also tried to limit bias when reviewing and testing my own code. Developer bias can be a concern because the person who writes the software already knows how it is supposed to work. Because of this, it can be easy to focus on successful scenarios and overlook ways the software could fail. I reduced this bias by returning to the written requirements and creating tests based on them instead of only relying on my own expectations. I intentionally tested null values, excessive character lengths, duplicate IDs, incorrectly formatted phone numbers, and invalid appointment dates. These tests forced me to look at the program from the perspective of someone trying to identify problems rather than only from the perspective of the person who created it. In a professional environment, I would also use practices such as peer reviews and having other

developers or testers examine my work. Another person may approach the software differently and identify an issue that the original developer overlooked. Combining independent reviews with automated testing would help reduce the possibility that assumptions made during development affect the quality of the final product.

Finally, this project showed me why discipline and commitment to quality are important in software engineering. Cutting corners when writing or testing code may save time in the short term, but defects that remain in an application can become more difficult and expensive to correct later. For example, failing to test duplicate IDs might seem like a small issue in a basic application, but in a larger system it could result in the wrong records being modified or deleted. A small validation problem can become a much larger problem once other components begin depending on incorrect data. As I continue working with software, I plan to avoid technical debt by testing new functionality as I develop it instead of waiting until the end of a project. I also plan to keep software requirements connected to my test cases, refactor duplicated or overly complicated code, use automated testing and coverage tools, and address defects when they are discovered rather than allowing them to build up. Maintaining these habits will make my code easier to maintain and reduce the possibility that small problems become larger ones later in development.

Project One helped me understand that software testing involves much more than making sure code runs without errors. Effective testing requires understanding the requirements, considering both expected and unexpected inputs, and creating tests that demonstrate whether the software behaves correctly. Through developing contact, task, and appointment services, I gained experience with JUnit 5, unit testing, positive and negative testing, boundary testing, and code coverage. Achieving approximately 90% coverage across my services gave me confidence

that the majority of the important code paths and requirements were being tested. More importantly, the project taught me the value of caution, limiting bias, and remaining disciplined about software quality. Testing my own code showed me why it is important to look beyond situations where the program works correctly and consider the ways it could fail. These practices will continue to be important as I work on larger and more complex software projects.

References

International Software Testing Qualifications Board. (n.d.). ISTQB glossary.

<https://glossary.istqb.org/>

JUnit Team. (n.d.). JUnit 5 user guide. <https://docs.junit.org/5.13.4/user-guide/>